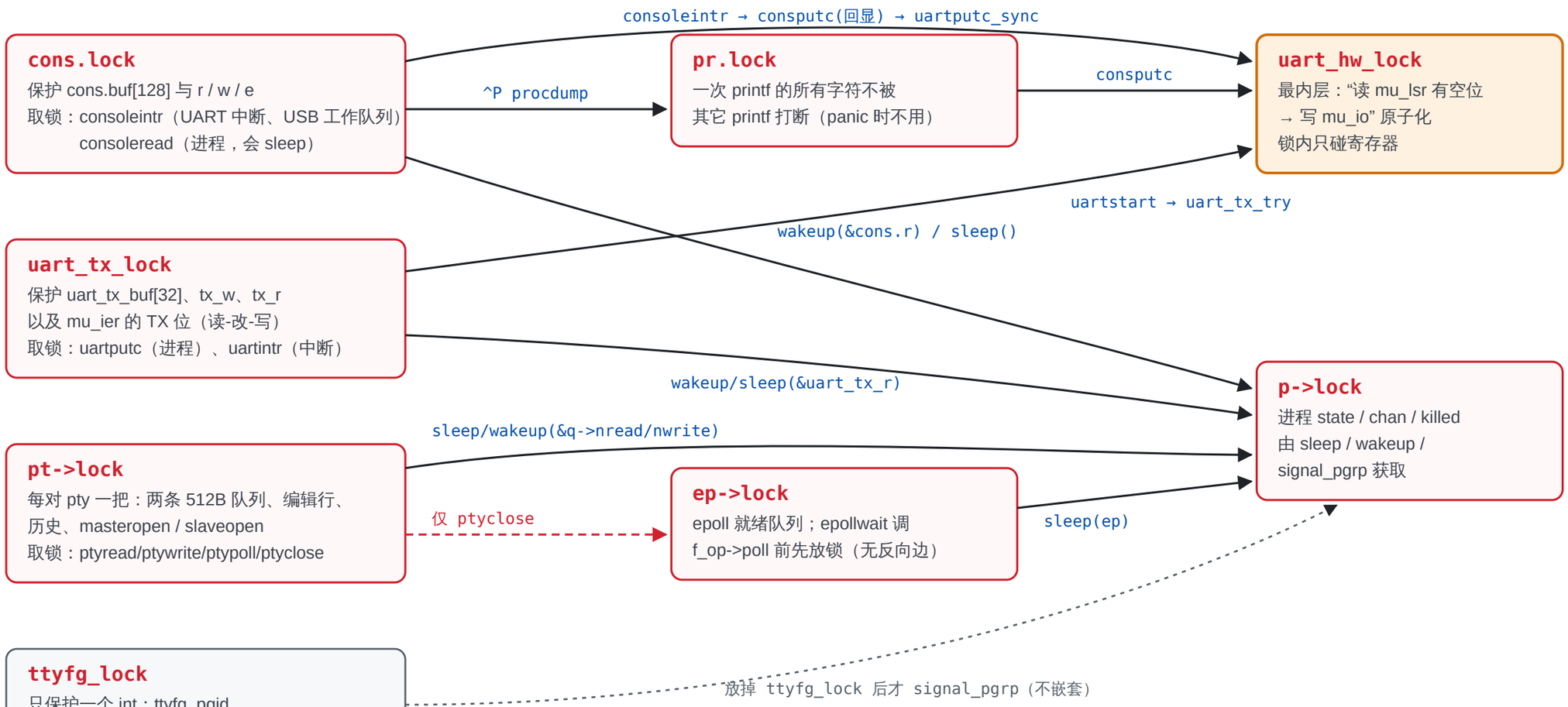
tty 相关锁的嵌套顺序（箭头 A → B：持有 A 时再获取 B）

外层：各对象自己的锁　　中间层　　内层



结论

- 图是有向无环图，按“外→中→内”取锁，不存在 AB-BA 死锁。
- 全部是 spinlock：acquire 先 push_off，所以中断处理函数拿 cons.lock 不会与本核进程自锁。
- ^C 的 ttyintr 在放掉 cons.lock 之后才调，避免 cons → ttyfg → p 三层嵌套，也缩短关中断的时间。
- 虚线红边是唯一不对称点，见笔记问题 6。

sleep / wakeup 通道（sleep(chan, lk) 在 p->lock 保护下放掉 lk，防止丢失唤醒）

chan	配对锁	谁睡（条件）	谁唤醒
`&cons.r`	`cons.lock`	consoleread：r == w（还没有完整一行）	consoleintr：收到 \n / ^D / 缓冲满，或 ^C 清空
`&uart_tx_r`	`uart_tx_lock`	uartputc：tx_w == tx_r + 32（环满）	uartstart：每搬走 1 字节（进程或 TX 中断里）
`&q->nread`	`pt->lock`	ptyread：队列空且对端仍打开	ptywrite / ptyinput 收到 \n / ptyqput 满时 / ptyclose
`&q->nwrite`	`pt->lock`	ptyqput：队列满（512）且读端仍打开	ptyread 读走数据后 / ptyclose
`(任意 chan)`	`p->lock`	上面任何一个睡眠者	signal_pgrp / kill：置 killed 后 make_runnable，读者醒来检查 killed 返回 -1